

Enlear Academy · [Follow publication](#)

AWS Encryption SDK

5 min read · Jan 25, 2020



Manoj Fernando

Follow



Listen



Share



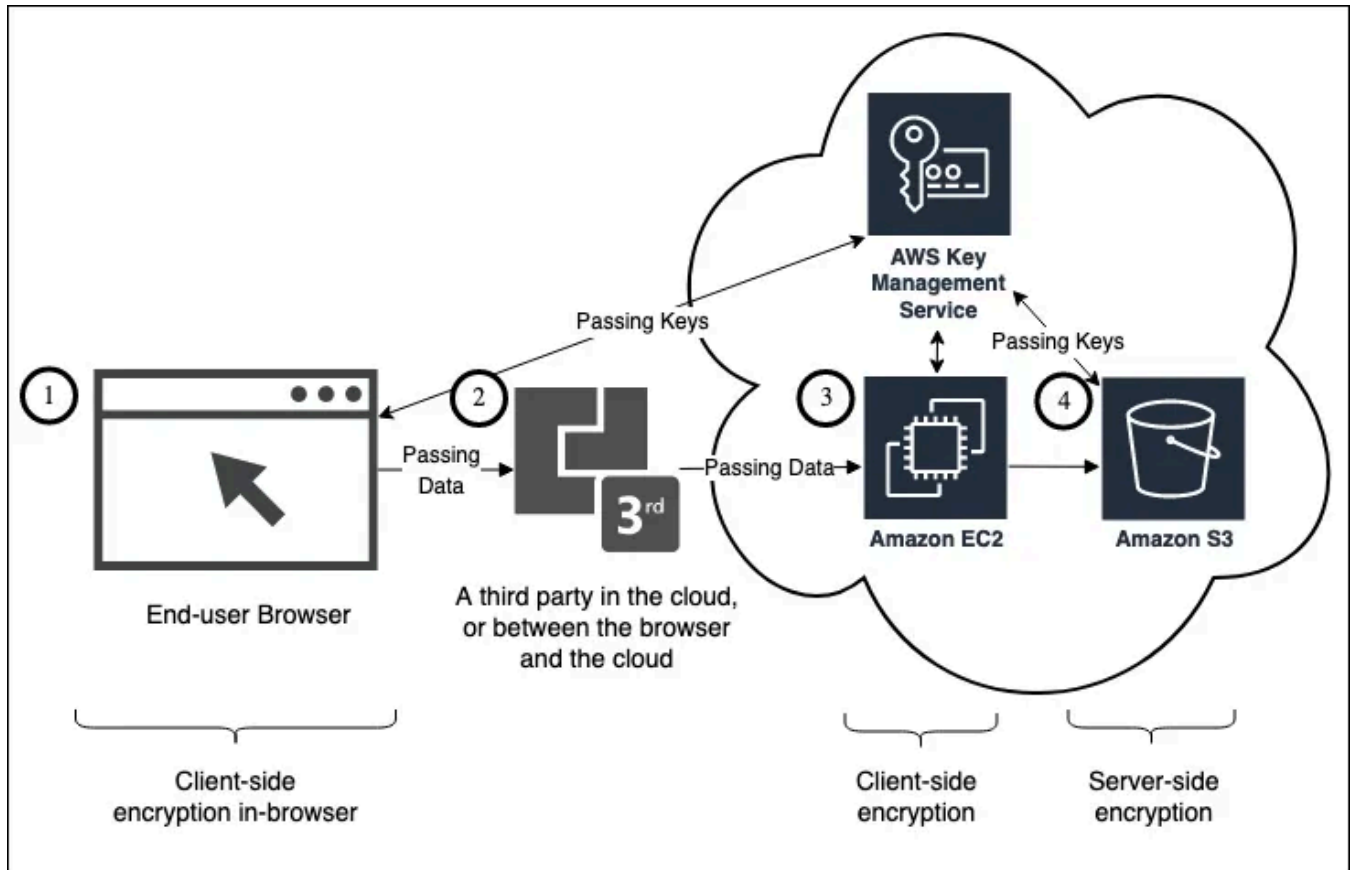
Photo by James Pond on Unsplash

This is part 03 of Data Encryption on the AWS series. You may find the related video for this blog post [here](#).

In the previous blog post, we discussed how to use OpenSSL with AWS KMS in order to encrypt/decrypt sensitive data. You can find the previous blog post [here](#). Today let us focus on **AWS Encryption SDK** with an example.

What is AWS Encryption SDK?

The AWS Encryption SDK is a client-side encryption library designed to make it easy for everyone to encrypt and decrypt data using industry standards and best practices. It enables you to focus on the core functionality of your application, rather than on how to best encrypt and decrypt your data.



Source — AWS Documentation

AWS Encryption SDK only requires us to provide it a **master key(s)** and then we can use straightforward methods to encrypt or decrypt our data at the client-side.

You don't have to worry about which encryption algorithm to use, how to maintain data-keys for the given master keys and how to ensure the data has not tampered between the time it is written and when it is read. AWS Encryption SDK handles everything for you. That makes our lives very easy!

At this point in time, AWS Encryption SDK supports JavaScript, C, Java and Python languages. We use JavaScript SDK for the example below.

When to use AWS Encryption SDK?

You can use Encryption SDK for Client-side encryption in browsers and other clients (e.g. EC2 instance that process data before storing in S3/Database). It is quite useful to encrypt data in distributed systems that communicate with different microservices and third party services.

However, AWS offers two other encryption clients for client-side encryption of data.

1. DynamoDB Encryption Client

2. S3 Encryption Client

AWS Encryption SDK and above encryption clients are not compatible. You cannot decrypt data that is encrypted by DynamoDB Encryption Client using AWS Encryption SDK. AWS recommends using above encryption clients if you are specifically working with DynamoDB or S3 as they provide additional functionalities suitable for DynamoDB or S3. For example, DynamoDB Encryption Client preserves partition/sort keys and encrypt only other data attributes.

AWS Encryption SDK can be used for general encryption workloads.

How to use AWS Encryption SDK?

AWS Encryption SDK provides SDKs for various programming languages. If you JavaScript SDK to encrypt data in the browser, you can use any other SDKs (e.g. Java, NodeJS) to decrypt at the server-side.

Get Manoj Fernando's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Following is an example of how you can use the Encryption SDK in NodeJS.

Step 01 — Setting up Encryption SDK for NodeJS

First of all, you need to install the AWS Encryption SDK for NodeJS.

```
npm install @aws-crypto/client-node
```

Then require KmsKeyring, encrypt and decrypt methods from the SDK

```
const { KmsKeyringNode, encrypt, decrypt } = require("@aws-crypto/client-node");
```

This example uses KMS as the Key management infrastructure. So we have to use KMS Keyring. As the names suggest, encrypt and decrypt methods are used to encrypt and decrypt data with datakeys generated by the keyring.

Note

AWS Encryption SDK for JavaScript uses Keyring to perform Envelope Encryption that is encrypting data keys with the master keys in KMS. You need to provide a reference to the Master Key and then the Keyring will create and manage datakeys to encrypt and decrypt data.

Step 02 — Configuring KMSKeyring with a CMK

Now that we have required the KMSKeyring, let's configure it using a CMK (Customer Master Key) created in AWS KMS. You can provide the **Arn** of the CMK to configure with the keyring.

```
const masterKeyId = "arn:aws:kms:us-east-1:123456:key/beece-abce-..";  
const keyring = new KmsKeyringNode({ masterKeyId });
```

(You can also provide multiple master keys in order to encrypt a datakey multiple times for better security as well)

Step 03 — Creating an Encryption Context

You can optionally create an encryption context for your plain text sensitive data in order to verify if the data has been tampered by anybody at decryption. (It is recommended to create an encryption context).

First, create a context with any useful metadata. Then pass the context to the encrypt method together with the plaintext sensitive data.

```
let plainText = "My passwords for sensitive data";

const context = {
  accountId: "100",
  purpose: "youtube demo",
  country: "Sri Lanka"
};
```

Step 04 — Encryption & Decryption

Now that we have the encryption context, we can start encrypting the data with Encryption SDK.

```
let plainText = "My passwords for sensitive data";

const { result } = await encrypt(keyring, plainText, {
  encryptionContext: context });
```

We use the encrypt method from the SDK and pass the keyring, plaintext and the encryption context as the parameters. As a result, we get the encryption version of the plaintext sensitive data.

We can decrypt this encrypted data in a different microservice in the distributed system by using the decrypt function of the SDK. The decrypt function expects the encrypted data and the keyring as the parameters.

```
const { plaintext, messageHeader } = await decrypt(keyring,
  encryptedData);
```

It will return the plaintext data and the “**messageHeader**” as a result of decrypt call. The messageHeader contains the encryption context we added at the point of encryption. So now we can verify that in order to make sure the data has not been tempered.

Step 05 — Verifying Encryption Context

Let's compare the original encryption context and the context returned by the decrypt call.

```
let originalContext = {
  accountId: "100",
  purpose: "youtube demo",
  country: "Sri Lanka"
};

Object.entries(originalContext).forEach(([key, value]) => {

  if (messageHeader.encryptionContext[key] === value) {
    console.log("Awesome. It is matching!");
  }
},
```

[Open in app](#) ↗[Sign up](#)[Sign in](#)**Medium** Search

If all the original context attributes are matching, we can conclude that the encrypted data is intact.

References

<https://docs.aws.amazon.com/encryption-sdk/latest/developer-guide/introduction>

AWS

Security

Encryption

[Follow](#)

Published in Enlear Academy

1.4K followers · Last published Nov 15, 2025

We provide high quality content on web development and cloud technologies for developers.

[Follow](#)

Written by Manoj Fernando

837 followers · 118 following